

Basic Pandas Concepts

Some very basic Pandas and python concepts to review.

Import the pandas package

```
import pandas as pd
```

Create a simple DataFrame

- syntax: `pd.DataFrame({column1: value1, column2: value2, column3: value3})`

You can have anything as column names and anything as values.

The only requirement is to have all value lists being of equal length (all are of length 3 in this example)

There are many ways to create a data frame and you will see some more during the course. All of them can be seen documented [here](#).

```
df = pd.DataFrame({'name': ['Bob', 'Jen', 'Tim'],
                  'age': [20, 30, 40],
                  'pet': ['cat', 'dog', 'bird']})

df
```

```
{
  "summary": {
    "\n  \"name\": \"df\",
    \"rows\": 3,
    \"fields\": [
      {
        \"column\": \"name\",
        \"properties\": {
          \"dtype\": \"string\",
          \"num_unique_values\": 3,
          \"samples\": [
            \"Bob\",
            \"Jen\",
            \"Tim\"
          ],
          \"semantic_type\": \"\",
          \"description\": \"\"
        }
      },
      {
        \"column\": \"age\",
        \"properties\": {
          \"dtype\": \"number\",
          \"std\": 10,
          \"min\": 20,
          \"max\": 40,
          \"num_unique_values\": 3,
          \"samples\": [
            20,
            30,
            40
          ],
          \"semantic_type\": \"\",
          \"description\": \"\"
        }
      },
      {
        \"column\": \"pet\",
        \"properties\": {
          \"dtype\": \"string\",
          \"num_unique_values\": 3,
          \"samples\": [
            \"cat\",
            \"dog\",
            \"bird\"
          ],
          \"semantic_type\": \"\",
          \"description\": \"\"
        }
      }
    ]
  },
  \"type\": \"dataframe\",
  \"variable_name\": \"df\"
}
```

View the column names and index values

The index is one of the most important concepts in pandas.

Each dataframe has only a single index which is always available as `df.index` and if you do not supply one (as we did not for this dataframe) a new one is made automatically.

Indexes define how to access rows of the dataframe.

The simplest index is the range index but there are more complex ones like interval index, datetime index and multi index.

We will explore indexes more in depth during the course of this lecture.

```
print(df.columns)
print(df.index)

Index(['name', 'age', 'pet'], dtype='object')
RangeIndex(start=0, stop=3, step=1)
```

Select a column by name in 2 different ways

These two ways are equivalent and can be used interchangeably almost always.

The primary exception is when the name of the column contains spaces. If for example we had a column called "weekly sales" we have to use `df['weekly sales']` because `df.weekly sales` is a syntactic error.

```
print(df['name'])
print(df.name)

0    Bob
1    Jen
2    Tim
Name: name, dtype: object
0    Bob
1    Jen
2    Tim
Name: name, dtype: object
```

Select multiple columns

To select multiple columns we use `df[columns_to_select]` where `columns_to_select` are the columns we are interested in given as a simple python list. As the result we will get another data frame.

This is the equivalent of listing columns names in **SELECT** part of a sql query.

```
df[['name', 'pet']]

{"summary": "{\n  \"name\": \"df[['name', 'pet']]\", \n  \"rows\": 3, \n  \"fields\": [\n    {\n      \"column\": \"name\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 3, \n        \"samples\": [\n          \"Bob\", \n          \"Jen\", \n          \"Tim\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    }, \n    {\n      \"column\": \"pet\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 3, \n        \"samples\": [\n          \"cat\", \n          \"dog\", \n          \"\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    } \n  ], \n  \"columns\": [\n    \"name\", \n    \"pet\" \n  ] \n}"}
```

```
\ "bird\" \n      ], \n      \ "semantic_type\" : \ "\", \n      \ "description\" : \ "\", \n      } \n      } \n      ] \n }", "type": "dataframe" }
```

Select a row by index

Regular selection of rows goes via its index. When using range indices we can access rows using integer indices but this will not work when using datetime index for example.

We can always access any row in the dataframe using `.iloc[i]` for some integer `i`.

The result is a series object from which we can access values by using column indexing.

```
df.iloc[0]
```

```
name      Bob
age       20
pet       cat
Name: 0, dtype: object
```

Sort Function

- pandas.pydata.org
- https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.sort_values.html

Sort the data by pet

There are two ways to sort.

- By index
- By value

By value means sorting according to value in a column.

In this example we sort the rows of the dataframe based on values in 'pet' column.

The parameter `ascending = True` means that we want the rows sorted in ascending order. This is the same as sql 'ASC'. To get descending order use `ascending = False`.

`inplace` is very important and you should always remember it. When `inplace=True` the dataframe is modified in place which means that no copies are made and your previous data stored in the dataframe is lost. By default `inplace` is always `False`. When it is `false` a copy is made of your data and that copy is sorted and returned as output.

The output of `sort_values` is always a dataframe returned but the behaviour depends strongly on the `inplace` parameter.

```
df.sort_values('pet', inplace=True, ascending=True)
```

Indexing with DataFrames

Everything we discussed about indexing in numpy arrays applies to dataframes as well.

DataFrames are very similar to 2d-arrays with the main exception being that in DataFrames you can index using strings (column names).

View the index after the sort

```
df
{"summary":{"\n  \"name\": \"df\",\n  \"rows\": 3,\n  \"fields\": [\n    {\n      \"column\": \"name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 3,\n        \"samples\": [\n          \"Tim\",\n          \"Bob\",\n          \"Jen\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"age\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 10,\n        \"min\": 20,\n        \"max\": 40,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          40,\n          20,\n          30\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"pet\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 3,\n        \"samples\": [\n          \"bird\",\n          \"cat\",\n          \"dog\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  }}, \"type\": \"dataframe\", \"variable_name\": \"df\"}
```

Difference between loc and iloc

- `.loc` selection is based on the value of the index. For example if the index was categorical we could index via some category.
- `.iloc` selection is **always** based on integer positions. When using `iloc` we are treating the dataframe as 2d-array with no special structure compared to the case of `.loc`

```
df.loc[0] #index based
```

```
name    Bob
age      20
pet     cat
Name: 0, dtype: object
```

```
df.iloc[0] #relative position based indexing
```

```
name    Tim
age      40
pet     bird
Name: 2, dtype: object
```

Use `iloc` to select all rows of a column

This will select all rows of the second column.

Remember `:` = `::1`

First index is always row and second is always column when dealing with dataframes.

```
df.iloc[:,2]
```

```
2    bird
0     cat
1     dog
Name: pet, dtype: object
```

Use `iloc` to select the last row

```
df.iloc[-1,:]
```

```
name    Jen
age      30
pet     dog
Name: 1, dtype: object
```

Basic Pandas Functionality

Before we learn about what Pandas can do, we need to first import some data

Importing Data

Python allows you to connect to any type of database. To make this easy for newbies, we've create a notebook to help you connect to the Strata Scratch platform and pull data. Use the notebook below to pull data from our database.

[Connect to Strata Scratch with Python](#)

Install the Database Module

The code below installs a postgres database module to allow our notebook to connect to the Strata Scratch database

```
!pip install psycopg2
```

```
Requirement already satisfied: psycopg2 in
/usr/local/lib/python3.12/dist-packages (2.9.11)
```

Import Required Modules

Import a few required modules that enables us to query data and perform analytics

```
import numpy as np
import pandas as pd
import psycopg2 as ps
```

Connect to Strata Scratch

Make sure to enter your username and database password. Your database password is not the same as your login password. You can find your database password in the Profile tab once logged into Strata Scratch.

Pull the Titanic Dataset From Strata Scratch

```
import kagglehub
path = kagglehub.dataset_download("yasserh/titanic-dataset")
print(path)
```

Using Colab cache for faster access to the 'titanic-dataset' dataset.
/kaggle/input/titanic-dataset

```
import pandas as pd
import os
```

```
csv_file = os.path.join(path, "Titanic-Dataset.csv")
df = pd.read_csv(csv_file)
```

Enter SQL code below to pull the dataset you're interested in

If you get an error, it likely means that the connection timed out. Try connecting to Strata Scratch again before executing the code below.

A list of datasets is found in SQL LAB in Strata Scratch.

```
import sqlite3
conn = sqlite3.connect("titanic.db")
df.to_sql("titanic", conn, if_exists="replace", index=False)

cur = conn.cursor()

cur.execute(""" SELECT * FROM titanic;""")

data = cur.fetchall()
colnames = [desc[0] for desc in cur.description]

data = pd.DataFrame(data, columns=colnames)

cur.close()
conn.close()
```

Check To See If Your Pulled The Dataset

The Titanic dataset should be in a pandas dataframe named `data`

```
data.head()
```

```
{
  "summary": {
    "name": "data",
    "rows": 891,
    "fields": [
      {
        "column": "PassengerId",
        "properties": {
          "dtype": "number",
          "std": 257,
          "min": 1,
          "max": 891,
          "num_unique_values": 891,
          "samples": [
            710,
            440,
            841
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "Survived",
        "properties": {
          "dtype": "number",
          "std": 0,
          "min": 0,
          "max": 1,
          "num_unique_values": 2,
          "samples": [
            1,
            0
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "Pclass",
        "properties": {
          "dtype": "number",
          "std": 0,
          "min": 1,
          "max": 3,
          "num_unique_values": 3,
          "samples": [
            3,
            1
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "Name",
        "properties": {
          "dtype": "string",
          "num_unique_values": 891,
          "samples": [
            "Moubarek, Master. Halim Gonios (William George)",
            "Kvillner, Mr. Johan Henrik Johannesson"
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "Sex",
        "properties": {
          "dtype": "category",
          "num_unique_values": 2,
          "samples": [
            "female",
            "male"
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "Age",
        "properties": {
          "dtype": "number",
          "std": 14.526497332334044,
          "min": 0.42,
          "max": 80.0,
          "num_unique_values": 88,
          "samples": [
            0.75,
            22.0
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "SibSp",
        "properties": {
          "dtype": "number",
          "std": 1,
          "min": 0,
          "max": 8,
          "num_unique_values": 7,
          "samples": [
            1,
            0
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "Parch",
        "properties": {
          "dtype": "number",
          "std": 0,
          "min": 0,
          "max": 6,
          "num_unique_values": 7,
          "samples": [
            0,
            1
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "Ticket",
        "properties": {
          "dtype": "string",
          "num_unique_values": 681,
          "samples": [
            "11774",
            "248740"
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "Fare",
        "properties": {
          "dtype": "number",
          "std": 49.693428597180905,

```



```

}\n    },\n    {\n        \"column\": \"Sex\", \n        \"properties\": {\n            \"dtype\": \"category\", \n            \"num_unique_values\": 2, \n            \"samples\": [\n                \"female\", \n                \"male\" \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    }, \n    {\n        \"column\": \"Age\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 14.526497332334044, \n            \"min\": 0.42, \n            \"max\": 80.0, \n            \"num_unique_values\": 88, \n            \"samples\": [\n                0.75, \n                22.0 \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    }, \n    {\n        \"column\": \"SibSp\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 1, \n            \"min\": 0, \n            \"max\": 8, \n            \"num_unique_values\": 7, \n            \"samples\": [\n                1, \n                0 \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    }, \n    {\n        \"column\": \"Parch\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 0, \n            \"min\": 0, \n            \"max\": 6, \n            \"num_unique_values\": 7, \n            \"samples\": [\n                0, \n                1 \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    }, \n    {\n        \"column\": \"Ticket\", \n        \"properties\": {\n            \"dtype\": \"string\", \n            \"num_unique_values\": 681, \n            \"samples\": [\n                \"11774\", \n                \"248740\" \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    }, \n    {\n        \"column\": \"Fare\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 49.693428597180905, \n            \"min\": 0.0, \n            \"max\": 512.3292, \n            \"num_unique_values\": 248, \n            \"samples\": [\n                11.2417, \n                51.8625 \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    }, \n    {\n        \"column\": \"Cabin\", \n        \"properties\": {\n            \"dtype\": \"category\", \n            \"num_unique_values\": 147, \n            \"samples\": [\n                \"D45\", \n                \"B49\" \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    }, \n    {\n        \"column\": \"Embarked\", \n        \"properties\": {\n            \"dtype\": \"category\", \n            \"num_unique_values\": 3, \n            \"samples\": [\n                \"S\", \n                \"C\" \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    } \n    ] \n}, \n\"type\": \"dataframe\", \n\"variable_name\": \"data\"}

```

Investigate the last 10 rows of data

tail is similar to head except it prints the last `n` rows.

```
data.tail(10)
```

```

{"summary": "{\n    \"name\": \"data\", \n    \"rows\": 10, \n    \"fields\": [\n        {\n            \"column\": \"PassengerId\", \n            \"properties\": {\n                \"dtype\": \"number\", \n                \"std\": 3, \n                \"min\":

```

```

882,\n          \"max\": 891,\n          \"num_unique_values\": 10,\n\"samples\": [\n          890,\n          883,\n          887\n],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n}\n    },\n    {\n        \"column\": \"Survived\",\n        \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 0,\n            \"min\": 0,\n            \"max\": 1,\n            \"num_unique_values\": 2,\n            \"samples\": [\n                1,\n                0\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n        },\n        {\n            \"column\": \"Pclass\",\n            \"properties\": {\n                \"dtype\": \"number\",\n                \"std\": 0,\n                \"min\": 1,\n                \"max\": 3,\n                \"num_unique_values\": 3,\n                \"samples\": [\n                    3,\n                    2\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n        },\n        {\n            \"column\": \"Name\",\n            \"properties\": {\n                \"dtype\": \"string\",\n                \"num_unique_values\": 10,\n                \"samples\": [\n                    \"Behr, Mr. Karl Howell\",\n                    \"Dahlberg, Miss. Gerda Ulrika\"\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n        },\n        {\n            \"column\": \"Sex\",\n            \"properties\": {\n                \"dtype\": \"category\",\n                \"num_unique_values\": 2,\n                \"samples\": [\n                    \"female\",\n                    \"male\"\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n        },\n        {\n            \"column\": \"Age\",\n            \"properties\": {\n                \"dtype\": \"number\",\n                \"std\": 6.050711620223782,\n                \"min\": 19.0,\n                \"max\": 39.0,\n                \"num_unique_values\": 9,\n                \"samples\": [\n                    26.0,\n                    22.0\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n        },\n        {\n            \"column\": \"SibSp\",\n            \"properties\": {\n                \"dtype\": \"number\",\n                \"std\": 0,\n                \"min\": 0,\n                \"max\": 1,\n                \"num_unique_values\": 2,\n                \"samples\": [\n                    1,\n                    0\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n        },\n        {\n            \"column\": \"Parch\",\n            \"properties\": {\n                \"dtype\": \"number\",\n                \"std\": 1,\n                \"min\": 0,\n                \"max\": 5,\n                \"num_unique_values\": 3,\n                \"samples\": [\n                    0,\n                    5\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n        },\n        {\n            \"column\": \"Ticket\",\n            \"properties\": {\n                \"dtype\": \"string\",\n                \"num_unique_values\": 10,\n                \"samples\": [\n                    \"111369\",\n                    \"7552\"\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n        },\n        {\n            \"column\": \"Fare\",\n            \"properties\": {\n                \"dtype\": \"number\",\n                \"std\": 9.96829415867574,\n                \"min\": 7.05,\n                \"max\": 30.0,\n                \"num_unique_values\": 9,\n                \"samples\": [\n                    23.45,\n                    10.5167\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n        },\n        {\n            \"column\": \"Cabin\",\n            \"properties\": {\n                \"dtype\": \"category\",\n                \"num_unique_values\":

```

```
2,\n      \"samples\": [\n          \"C148\", \n          \"B42\", \n      ], \n      \"semantic_type\": \"\", \n      \"description\": \"\" \n  }, \n  {\n      \"column\": \"Embarked\", \n      \"properties\": {\n          \"dtype\": \"category\", \n          \"num_unique_values\": 3, \n          \"samples\": [\n              \"S\", \n              \"Q\", \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n      } \n  } \n  ] \n} \", \"type\": \"dataframe\"}
```

Investigate the data types in the DataFrame

This method will tell you the types of columns.

Types are automatically inferred by pandas and usually you do not have to worry about them.

[docs](#)

```
data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age         714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Get some summary statistics

To learn more about describe visit [this link](#)

```
data.describe().T

{"summary": "{\n  \"name\": \"data\", \n  \"rows\": 7, \n  \"fields\": [\n    {\n      \"column\": \"count\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 66.89971172263323, \n        \"min\": 714.0, \n        \"max\": 891.0, \n        \"num_unique_values\": 2, \n        \"samples\": [\n          714.0, \n          891.0\n        ], \n        \"semantic_type\": \"\", \n      }\n    }\n  ]\n}
```

[illegible]

Filtering Dataframes

You can filter data based on the columns and values in the dataframe

Filter the data for men

There are two pieces of the puzzle here:

- `data.sex=='male'` will give a boolean array where True means that row has a column called sex which has value 'male'. This numpy array is called the predicate.

- `data[data.sex=='male']` will give back all rows for which the predicate holds true.

The result of this filter is a dataframe with same columns as the input dataframe.

```
data[data.sex=='male']

-----
-----
AttributeError                                Traceback (most recent call
last)
/tmp/ipython-input-4084614405.py in <cell line: 0>()
----> 1 data[data.sex=='male']

/usr/local/lib/python3.12/dist-packages/pandas/core/generic.py in
__getattr__(self, name)
    6297         ):
    6298             return self[name]
-> 6299         return object.__getattribute__(self, name)
    6300
    6301     @final

AttributeError: 'DataFrame' object has no attribute 'sex'
```

Filter the ages for the men

Again there are two important parts:

- `data.sex=='male'` is the predicate as before
- `data.age` means taking the values for the age column, and `data.age[data.sex=='male']` means taking all ages which are related to male rows.

The result of this is pandas series **not** a dataframe.

```
data.age[data.sex=='male']

-----
-----
AttributeError                                Traceback (most recent call
last)
/tmp/ipython-input-2879827074.py in <cell line: 0>()
----> 1 data.age[data.sex=='male']

/usr/local/lib/python3.12/dist-packages/pandas/core/generic.py in
__getattr__(self, name)
    6297         ):
    6298             return self[name]
-> 6299         return object.__getattribute__(self, name)
    6300
    6301     @final
```

```
AttributeError: 'DataFrame' object has no attribute 'age'
```

Adding methods to filters

A method is a function and is used frequently when analyzing data in Pandas. There are countless Pandas methods. We'll go over a few of the basic ones to show how you can use methods to quickly analyze your data.

How many men and women were on the Titanic?

The pipeline always goes the same way

- Predicate is evaluated
- Data is filtered according to a predicate
- An aggregate value is computed after the filtering.

The count method simply counts the number of frames in the dataframe.

```
data.sex[data.Sex=='male'].count()  
#fixed  
data['Sex'][data['Sex'] == 'male'].count()  
np.int64(577)  
  
data.sex[data.Sex=='male'].count()  
#fixed  
data['Sex'][data['Sex'] == 'female'].count()  
np.int64(314)
```

What was the survival rate for adult men (age>=18)

Here we combine predicates using the and operator (&).

This operator applies the logical and operation between elements at matching positions.

For example:

- `x = np.array([True, False, True, True])`
- `y = np.array([False, True, False, True])`
- will give `x & y = np.array([True & False, False & True, True & False, True & True])`.

In the following example we use the or combiner (|).

You can combine any two boolean numpy arrays as long as they have the same shape using the & and | operators.

Combining regular python lists this way does not work.

```
data['Survived'][(data['Sex'] == 'male') & (data['Age'] >= 18)].mean()
np.float64(0.17721518987341772)
```

What was the survival rate for women and children?

The mean method is the same as AVERAGE in SQL.

```
data['Survived'][(data['Sex'] == 'female') | (data['Age'] < 18)].mean()
```

Use groupby to compare the survival rates of men and women

The `groupby` method is one of the most important tools you will use in your day to day work.

It's main input parameter is either a string denoting a column name or a list of strings denoting a list of column names.

It's output is a GroupBy object which is very similar to a dataframe.

The operation of groupby is the same as SQL GROUPBY.

For more info see the [docs](#).

```
data.groupby('sex')['survived'].mean()

-----
-----
KeyError                                Traceback (most recent call
last)
/tmp/ipython-input-2253236484.py in <cell line: 0>()
----> 1 data.groupby('sex')['survived'].mean()

/usr/local/lib/python3.12/dist-packages/pandas/core/frame.py in
groupby(self, by, axis, level, as_index, sort, group_keys, observed,
dropna)
    9181         raise TypeError("You have to supply one of 'by'
and 'level'")
    9182
-> 9183         return DataFrameGroupBy(
    9184             obj=self,
    9185             keys=by,

/usr/local/lib/python3.12/dist-packages/pandas/core/groupby/groupby.py
in __init__(self, obj, keys, axis, level, grouper, exclusions,
selection, as_index, sort, group_keys, observed, dropna)
    1327
    1328         if grouper is None:
-> 1329             grouper, exclusions, obj = get_grouper(
    1330                 obj,
```

```

1331             keys,

/usr/local/lib/python3.12/dist-packages/pandas/core/groupby/grouper.py
in get_grouper(obj, key, axis, level, sort, observed, validate,
dropna)
    1041             in_axis, level, gpr = False, gpr, None
    1042         else:
-> 1043             raise KeyError(gpr)
    1044     elif isinstance(gpr, Grouper) and gpr.key is not None:
    1045         # Add key to exclusions

KeyError: 'sex'

```

Create a DataFrame with groupby

```

new = data.groupby(['Sex', 'Pclass'])['Survived', 'Age'].mean()
new

-----
-----
ValueError                                Traceback (most recent call
last)
/tmp/ipython-input-2383668958.py in <cell line: 0>()
----> 1 new = data.groupby(['Sex', 'Pclass'])['Survived', 'Age'].mean()
      2 new

/usr/local/lib/python3.12/dist-packages/pandas/core/groupby/generic.py
in __getitem__(self, key)
    1945         # if len == 1, then it becomes a SeriesGroupBy and
this is actually
    1946         # valid syntax, so don't raise
-> 1947         raise ValueError(
    1948             "Cannot subset columns with a tuple with more
than one element. "
    1949             "Use a list instead."

ValueError: Cannot subset columns with a tuple with more than one
element. Use a list instead.

```

Importing and Exporting Data with Pandas

Pandas has easy to use functions for importing and exporting different data types:

- CSV Files
- Excel Worksheets
- Queries from Databases

Strata Scratch notebooks will exclusively be import data from our platform so we will not be covering other import techniques.

More Basic Pandas Exercises

What was the average age of the survivors?

```
data['Survived'].mean()  
np.float64(0.3838383838383838)
```

What was the combined survival rate of both children (age less than 18) and seniors (age greater than 60)?

```
data['Survived'][(data['Age'] >= 60) & (data['Age'] <= 18)].count()  
np.int64(0)
```

Group by pClass and investigate average survival rate, age and fare

```
print(f"{data.groupby('Pclass')[['Survived', 'Age', 'Fare']].mean()}\n")  
#or  
print(f"Pclass 1: {data.loc[(data['Pclass'] == 1) & (data['Age'] >= 18), ['Survived', 'Age', 'Fare']].mean()}\n")  
print(f"Pclass 2: {data.loc[(data['Pclass'] == 2) & (data['Age'] >= 18), ['Survived', 'Age', 'Fare']].mean()}\n")  
print(f"Pclass 3: {data.loc[(data['Pclass'] == 3) & (data['Age'] >= 18), ['Survived', 'Age', 'Fare']].mean()}\n")
```

	Survived	Age	Fare
Pclass			
1	0.629630	38.233441	84.154687
2	0.472826	29.877630	20.662183
3	0.242363	25.140620	13.675550

```
Pclass 1: Survived    0.637931  
Age                40.031609  
Fare                86.574115  
dtype: float64
```

```
Pclass 2: Survived    0.413333  
Age                33.443333  
Fare                20.782833  
dtype: float64
```

```
Pclass 3: Survived    0.202166  
Age                29.608303  
Fare                11.056360  
dtype: float64
```

Create a CSV with the names and ages of the survivors and another CSV file with the names and ages of the deceased. Please refer to documentation (to_csv method) to complete the exercise.

[Documentation for the method](#)

```
#getting the data first with the condition of survivor or deceased and then saving it using to_csv
survivors = data.loc[data['Survived'] == 1, ['Name', 'Age']]
survivors.to_csv('survivors.csv', index=False)

deceased = data.loc[data['Survived'] == 0, ['Name', 'Age']]
deceased.to_csv('deceased.csv', index=False)

#or
#Getting the data by using query then saving it using to_csv
data.query('Survived == 1')[['Name', 'Age']].to_csv('survivors.csv',
index=False)
data.query('Survived == 0')[['Name', 'Age']].to_csv('deceased.csv',
index=False)

#show data
survivors.dropna(subset=['Age']).to_csv('survivors.csv', index=False)
deceased.dropna(subset=['Age']).to_csv('deceased.csv', index=False)
```